# BMIT3173 Integrative Programming

# ASSIGNMENT 202605

Student Name : Serena Lim Sze Kee

Student ID : 25WMR9770

Programme : Bachelor in Information Technology (Honours) (Information Security)

Tutorial Group : Group 4

System Title : LearnSync - Integrated Educational Resource and Collaborative Learning Portal

Chosen SDG : SDG 4 - Quality Education

Modules : Module 1 - Identity, Access & Digital Credentialing

## AI Tools Usage Policy & Disclosure

This assignment is governed by the **TAR UMT Policy for the Use of Artificial Intelligence (AI) (PO/111:26)** and is classified under the **YELLOW (Limited AI)** category. AI tools may be used only for the purposes listed below, not to produce the technical work, analysis, or original content being assessed.

**Mandatory submission:** Every student must complete and submit the **AI Usage Disclosure Form** (below) with the report, whether or not AI tools were used, by 6th September 2026. A report submitted without the Form is incomplete and may not be marked.

**Permitted (Yellow) uses of AI tools:**

- Language refinement (grammar, spelling, clarity) of text you wrote, without altering its technical meaning.
- Clarifying taught concepts (e.g. how a design pattern, web-service protocol, or attack works) to aid your understanding.
- Debugging help on code you wrote, where AI only locates or explains an error you then fix yourself.
- Brainstorming early ideas (e.g. SDG framings or topic angles), provided the chosen direction and its justification are your own.

**Prohibited uses of AI tools:**

- Generating the PHP code, design-pattern implementation, class diagrams, or web-service code being assessed.
- Producing core arguments or analysis, including the design-pattern justification, threat analysis, and secure-coding rationale.
- Uploading confidential, proprietary, or others' personal data to public AI tools (Sections 2.5 and 5 of the AI Policy).

You remain responsible for the accuracy and originality of your work; AI errors do not excuse incorrect content. The lecturer may ask you to explain or demonstrate any part to verify authorship. Undisclosed or prohibited AI use is a breach of academic integrity under Section 4 of the AI Policy. **Complete the Form below; if AI was used, list each use and cite the tool(s) in your References (APA 7th).**

### AI Usage Disclosure Form

**Declaration (tick one):**

- ☐ No AI tools were used in the preparation of this report.
- ☑ AI tools were used as declared in the table below.

| AI Tool Used (name & version) | Purpose / How It Was Used | Report Section(s) Affected |
|---|---|---|
| Grammarly (2026 Edition) | Used solely for language refinement, spelling verification, and grammatical sentence flow of my own written text without altering technical logic. | Sections 1, 2, 4.1, 4.3, 5.1 |

*If no AI tools were used, tick "No AI tools used". Non-disclosure breaches the AI Policy.*

I declare this Form is true and complete and that my AI use complied with the AI Policy and the Yellow conditions above.

Sign: _____Serena_____ Date: ____6/9/2026____

# Table of Contents

# 1. Introduction to the System

## 1.1 System Overview

LearnSync is a web based Learning Management System (LMS). Lecturers use it to upload notes, set quizzes, give assignments and mark student work. Students use it to study the notes, sit the quizzes, hand in assignments, watch their progress, and ask questions in a course forum.

The problem LearnSync solves is about proof. In most systems, the certificate a student earns is just a PDF file. Anyone can open that PDF in a word processor and change the marks. If an employer wants to check whether a certificate is real, they must email the college and wait days for a reply. Most employers do not bother.

LearnSync fixes this. Every certificate it creates has three things:

- A unique ID printed on it, such as LS-2026-A7F3D9K2.
- A QR code that anyone can scan with a phone.
- A hidden security code, called a hash, which proves that nobody has changed the record.

Anyone can check a certificate in a few seconds, for free, without needing an account.

## 1.2 The Sustainable Development Goal

LearnSync is built around United Nations Sustainable Development Goal 4: Quality Education (SDG 4).

What SDG 4 aims to do:

SDG 4 wants to make sure education is fair, good quality and available to everyone by 2030 (United Nations, 2015). Two of its targets matter here. Target 4.3 is about giving everyone fair access to affordable college and university education. Target 4.4 is about increasing the number of people who have real skills for getting a job.

## 1.3 System Contribution to SDG 4 & Scope

LearnSync supports SDG 4 in three ways:

1. Who benefits: Students who earn certificates, lecturers who teach and mark the courses, administrators who control who can do what, and outside people such as employers. The employers matter most here, because they can check a certificate without having an account.

2. Making checking free and instant: Every certificate gets a unique ID and a QR code linking to a public checking page. An employer scans the code and sees the answer straight away. Today, checking a qualification takes time and effort, so many small employers simply skip it. Students without good connections are hurt the most by this. LearnSync removes that problem.

3. Proving the certificate is genuine: Each certificate stores a security code called a SHA-256 hash. The system works this code out again every time someone checks the certificate. If anyone changes the record, even by editing the database directly, the two codes will not match, and the page shows TAMPERED instead of valid. So trust is not just assumed. It can be proven.

What the system does not do:

LearnSync does not replace official government accreditation. It only makes the certificates a college already gives out easy and instant to check.

# 2. Module Description

## 2.1 Scope of Module 1: Identity, Access & Digital Credentialing

As the developer responsible for Module 1 (Identity, Access & Digital Credentialing Module), I designed and implemented the invitation-controlled account provisioning pipeline, the database-driven role permission matrix resolved at runtime through Laravel Gates, the account lifecycle and login-security controls, the complete security audit trail, weighted per-course progress computation with historical snapshotting, the cryptographic certificate issuance and public verification subsystem, the administrator-configurable award rules engine, and the per-user notification inbox.

## 2.2 Functional Breakdown & Class Paths

### F1.1: Invitation-Controlled Account Provisioning

- Description: Nobody can sign up by themselves. Going to /register gives a 404 Not Found page. An account only exists after an admin sends an Invitation. The invitation holds the person's email, the role they will get, a secret token, and a date when it expires. The system rejects a token if it is unknown, already used, expired, or if the email already has an account. The role comes from the invitation and not from the sign up form, so a user cannot make themselves an admin by changing the form data.

- Class Paths:

    - Controller: app/Http/Controllers/Auth/InvitedRegistrationController.php (create, store)

    - Controller: app/Http/Controllers/InvitationController.php (index, store, bulkStore)

    - Model: app/Models/Invitation.php

    - View Template: resources/views/auth/register-invited.blade.php

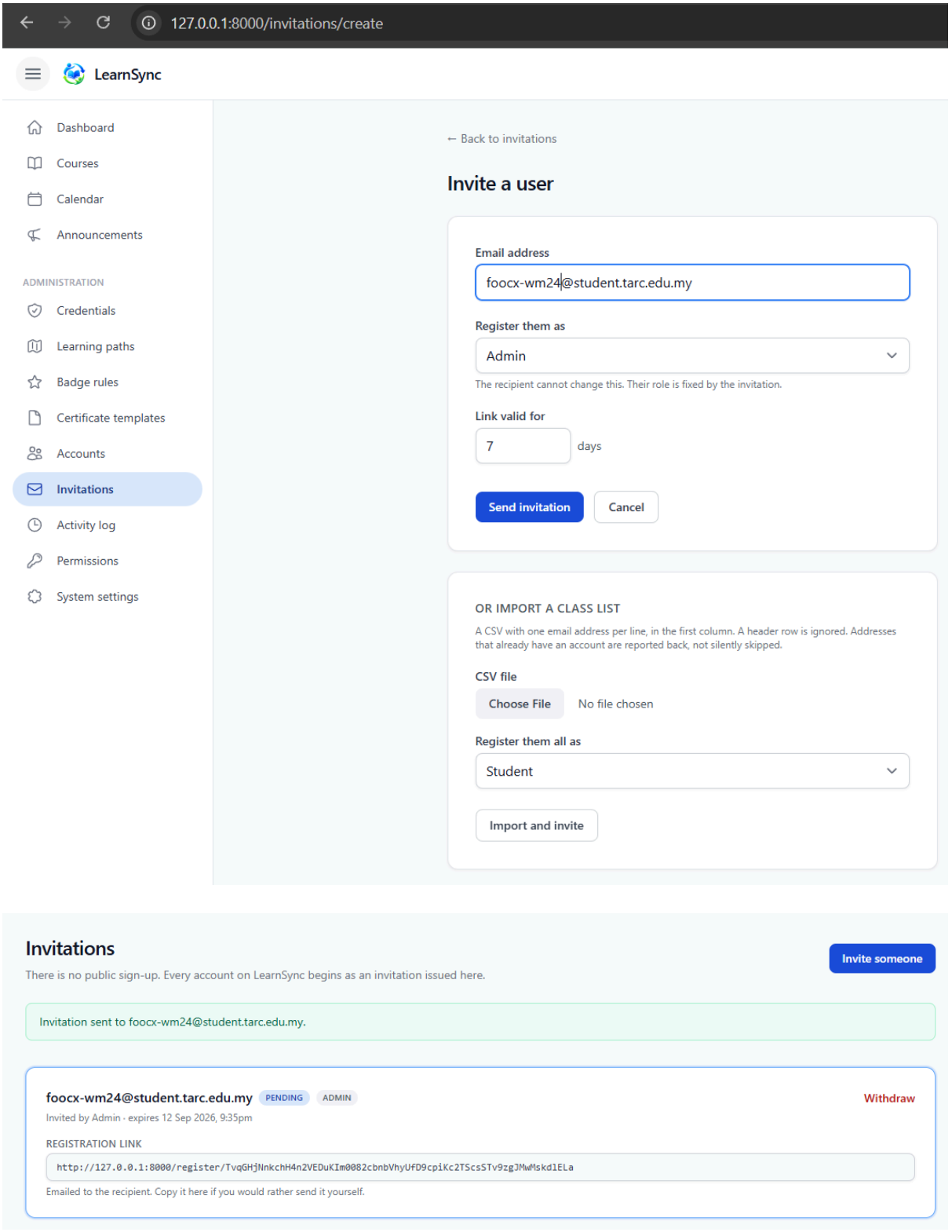



Figure 2.1: Administrator Invitation Issuance Screen with Tokenised Registration Link

### F1.2: Database-Driven Role Permission Matrix (RBAC)

- Description: The rules about who can do what are saved in the database as 35 permission keys, sorted into eight groups. The groups are Course Management, User and Access Management, System Configuration, Assessment, Forum, Credentialing, Progress and Profile. Each key is given to one or more roles. When the code asks $user->can('certificate.revoke'), Laravel checks these database tables. There is no line of code anywhere that says if the user is an admin. This means an admin can tick and untick boxes on a screen, and the change works on the very next page load, with no programming needed.
- Class Paths:
    - Controller: app/Http/Controllers/PermissionController.php (index, update)
    - Gate Registration: app/Providers/AppServiceProvider.php (registerPermissionGate)
    - Models: app/Models/Permission.php, app/Models/PermissionRole.php
    - View Template: resources/views/permissions/index.blade.php

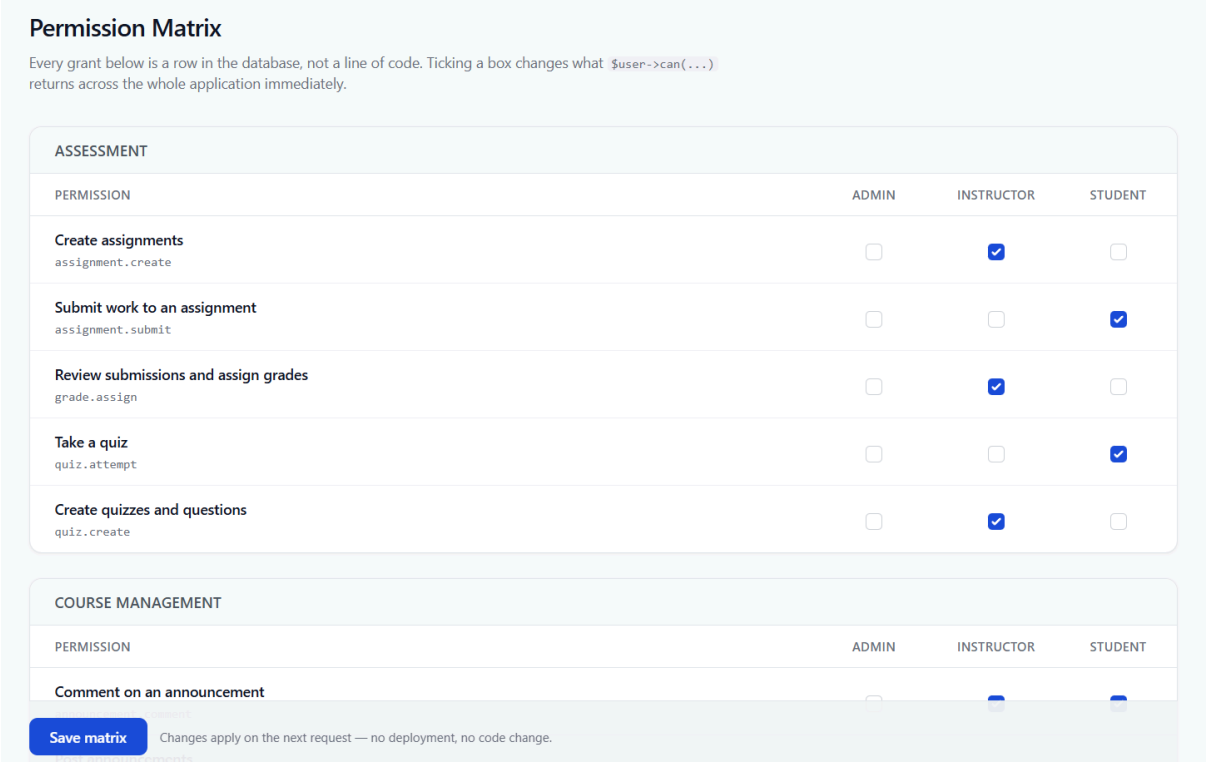
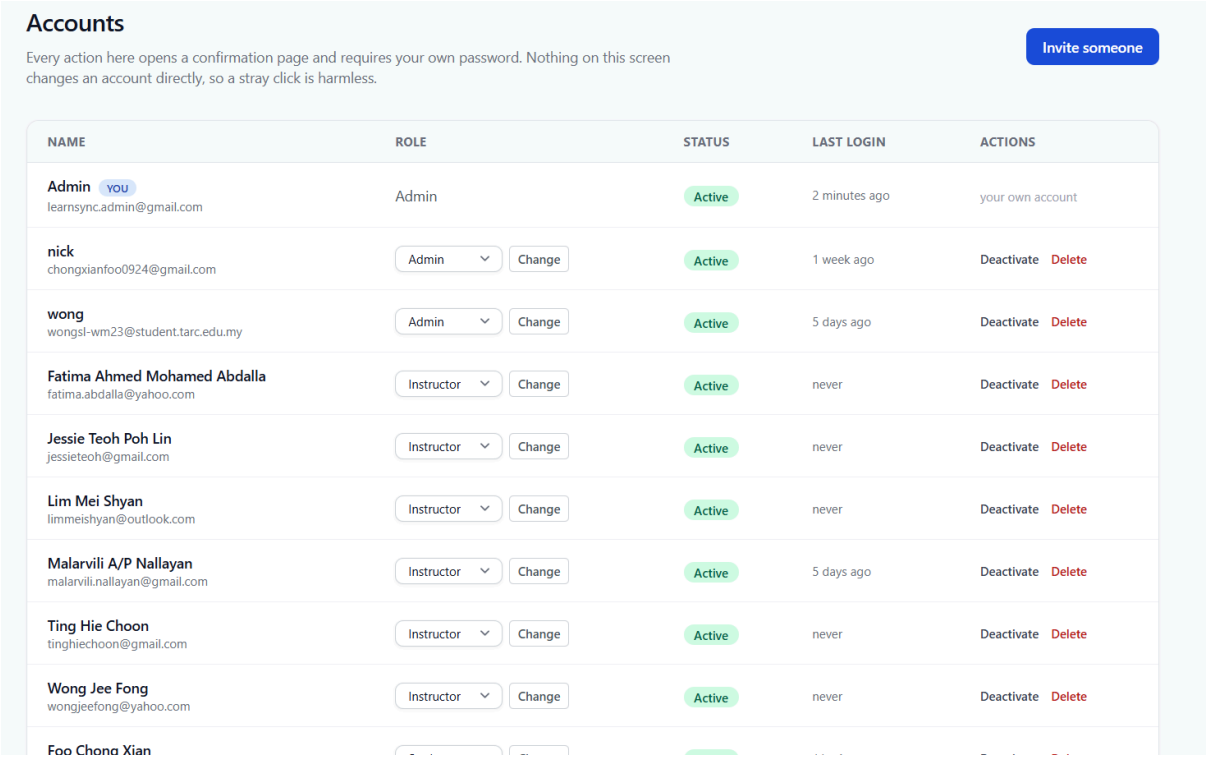



Figure 2.2: Runtime-Configurable Permission Matrix Grid

### F1.3: Account Lifecycle & Login Security Controls

- Description: Admins can turn accounts on and off, delete them, and unlock them. If someone gets the password wrong five times in a row, the account locks, and only an admin can unlock it. A new password cannot be the same as any of the last three. New users must change their password the first time they log in. Every dangerous action, such as deleting an account, asks the admin to type their own password on a separate page first. This means one accidental click can never change an account.
- Class Paths:
    - Controller: app/Http/Controllers/UserController.php (index, confirm, perform)
    - Middleware: app/Http/Middleware/EnsureAccountIsActive.php
    - Middleware: app/Http/Middleware/EnsurePasswordIsChanged.php
    - View Templates: resources/views/users/index.blade.php, resources/views/users/confirm.blade.php



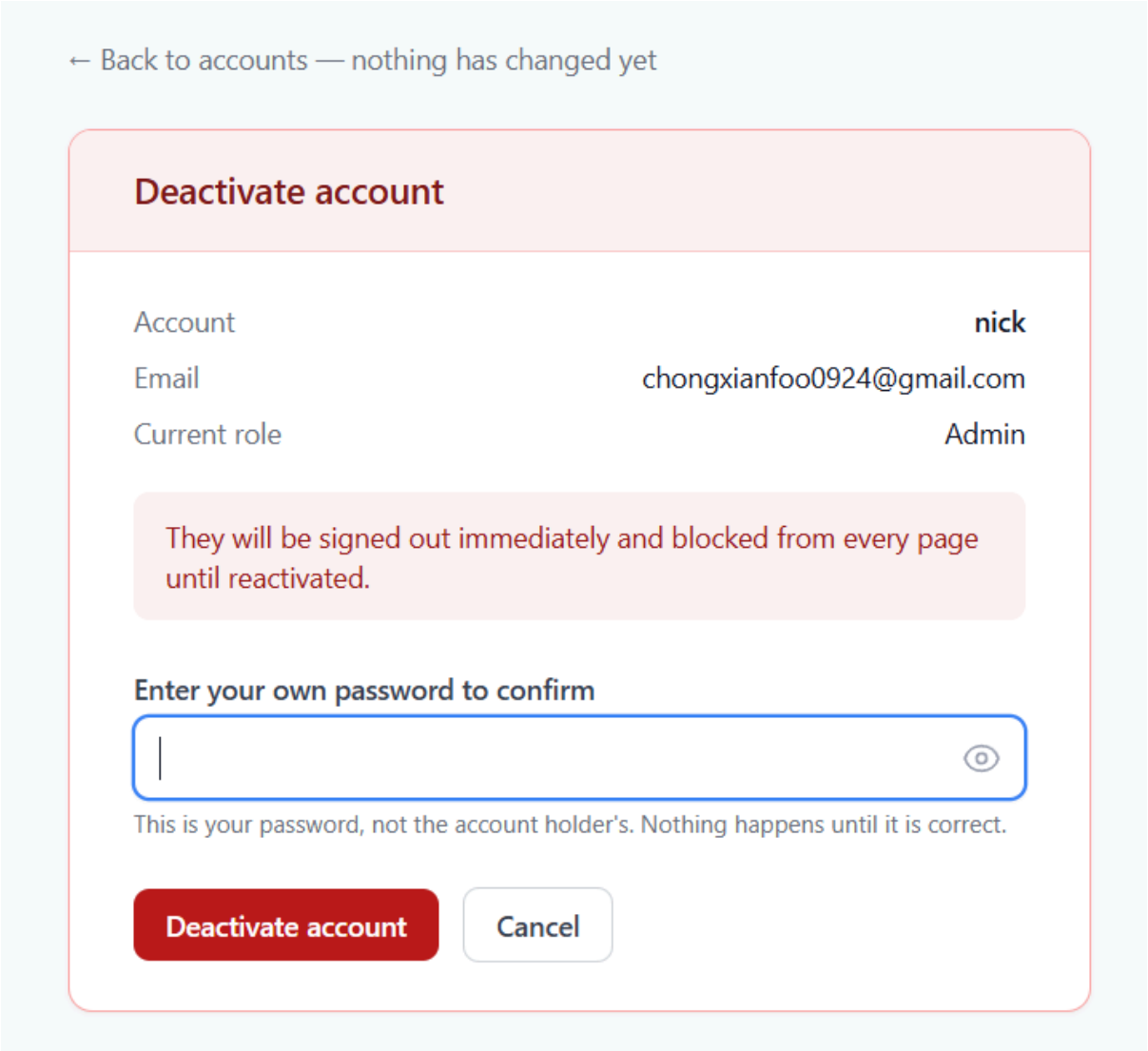



Figure 2.3: Account Management Screen and Password Re-Authentication Gate

### F1.4: Security Audit Trail with CSV Export

- Description: Every important action saves a row in the activity_logs table. Each row records who did it, what they did, what they did it to, their IP address, and their browser. The actions saved include auth.login, auth.logout, auth.failed, invitation.issued, user.registered, certificate.issued, certificate.revoked and course.student_removed. Instead of editing the login code, the system listens to Laravel's own login events. This means any new login method added later is recorded
- Class Paths:
    - Controller: app/Http/Controllers/ActivityLogController.php (index, export)
    - Model: app/Models/ActivityLog.php (record)
    - View Template: resources/views/activity_logs/index.blade.php

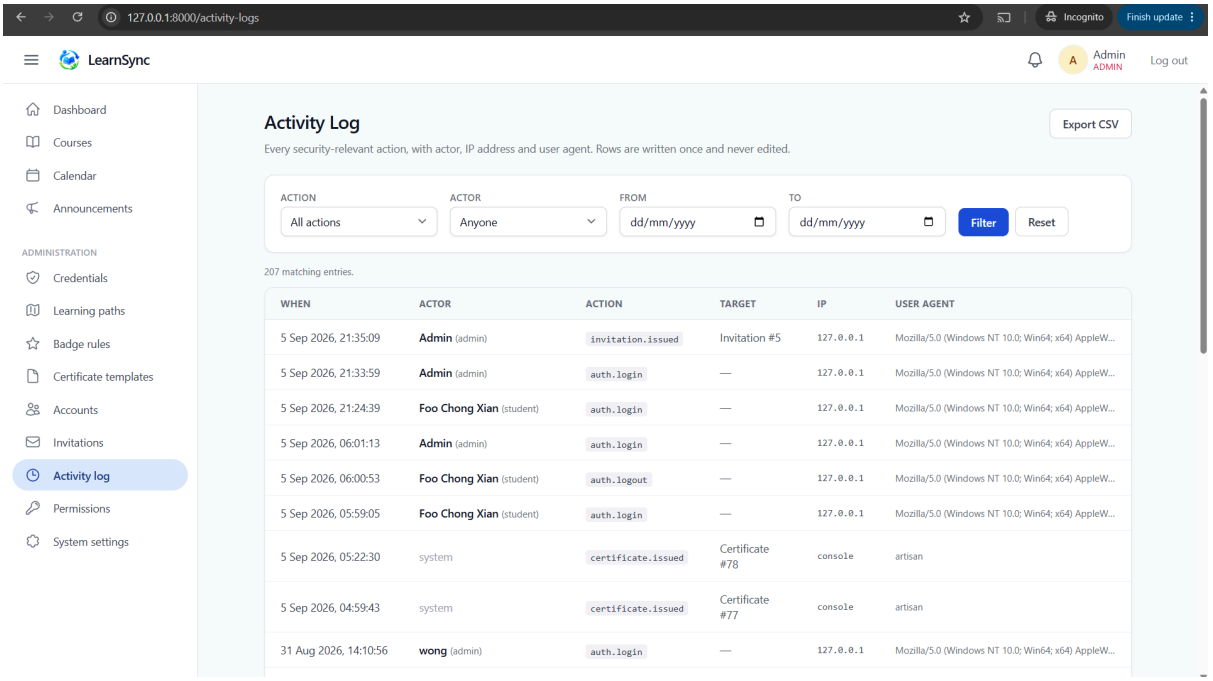



Figure 2.4: Filterable Security Audit Trail with CSV Export Control

### F1.5: Weighted Progress Computation & Historical Snapshotting

- Description: Progress is saved for each student in each course, not as one overall number. The system works it out from three things: quizzes passed, assignments handed in, and joining the course forum. The importance of each one, normally quiz 50%, assignment 40% and forum 10%, is stored in a settings table. This lets an admin change the balance without touching any code. Every time progress is recalculated, the system saves a snapshot. These snapshots are what let the student dashboard draw a line chart of progress over time.
- Class Paths:
    - Subsystem: app/Patterns/Facade/Subsystem/ProgressCalculator.php (recalculate, passThreshold)
    - Models: app/Models/StudentProgress.php, app/Models/ProgressSnapshot.php
    - View Template: resources/views/dashboard/student.blade.php

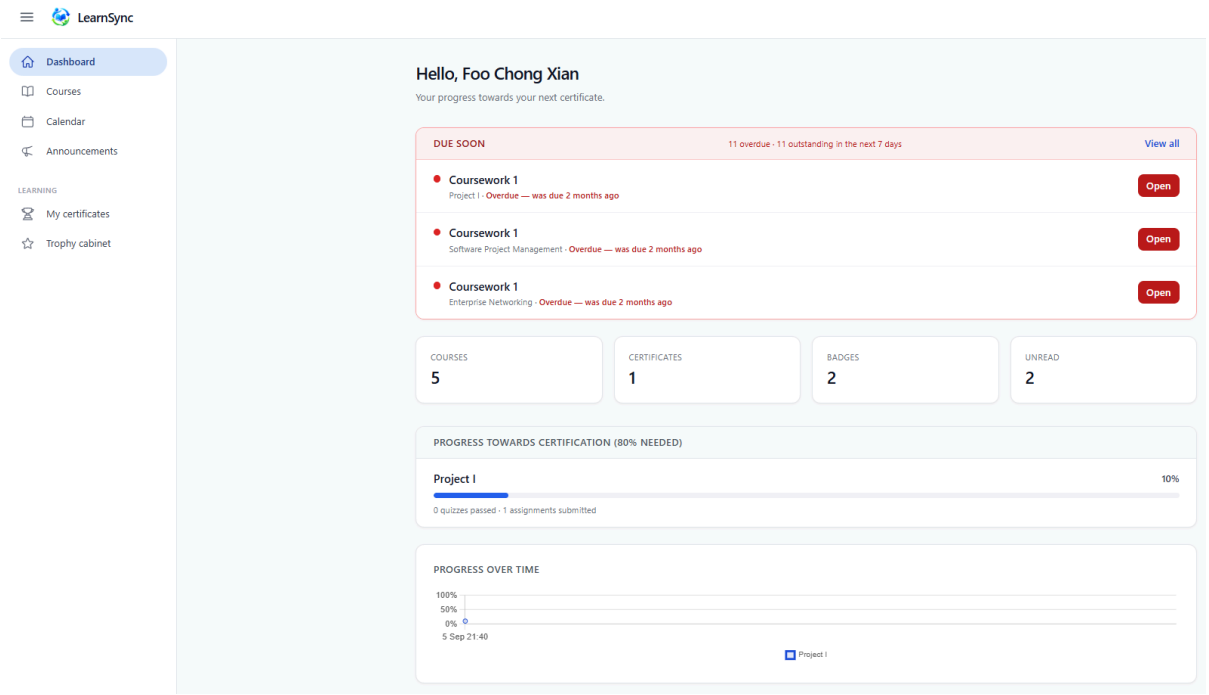



Figure 2.5: Student Progress-Over-Time Dashboard Chart

### F1.6: Cryptographic Certificate Issuance

- Description: When a student finishes a course, the system makes a certificate. It creates a unique ID using letters and numbers that are hard to mix up. The letters I, L, O and U are removed so that the digit 1 is never confused with I or L, and the digit 0 is never confused with O. It then adds a SHA-256 security code and makes a PDF with a QR code inside it.
- Two separate tests must both pass before a certificate is given:
    - The student's progress must reach the required percentage. This shows they did the work.
    - The student's average mark must be a pass. This shows they understood the work.
- Only checking progress was not enough. A hard working student who understood very little could still reach 80% just by joining in. That produced certificates showing a failing average mark, which a real certificate must never do.
- Class Paths:
    - Facade: app/Patterns/Facade/CredentialAuthority.php (issueCertificate, issuePathwayCertificate)
    - Subsystem: app/Patterns/Facade/Subsystem/CredentialIdGenerator.php
    - Subsystem: app/Patterns/Facade/Subsystem/CertificateRenderer.php
    - View Template: resources/views/certificates/pdf.blade.php

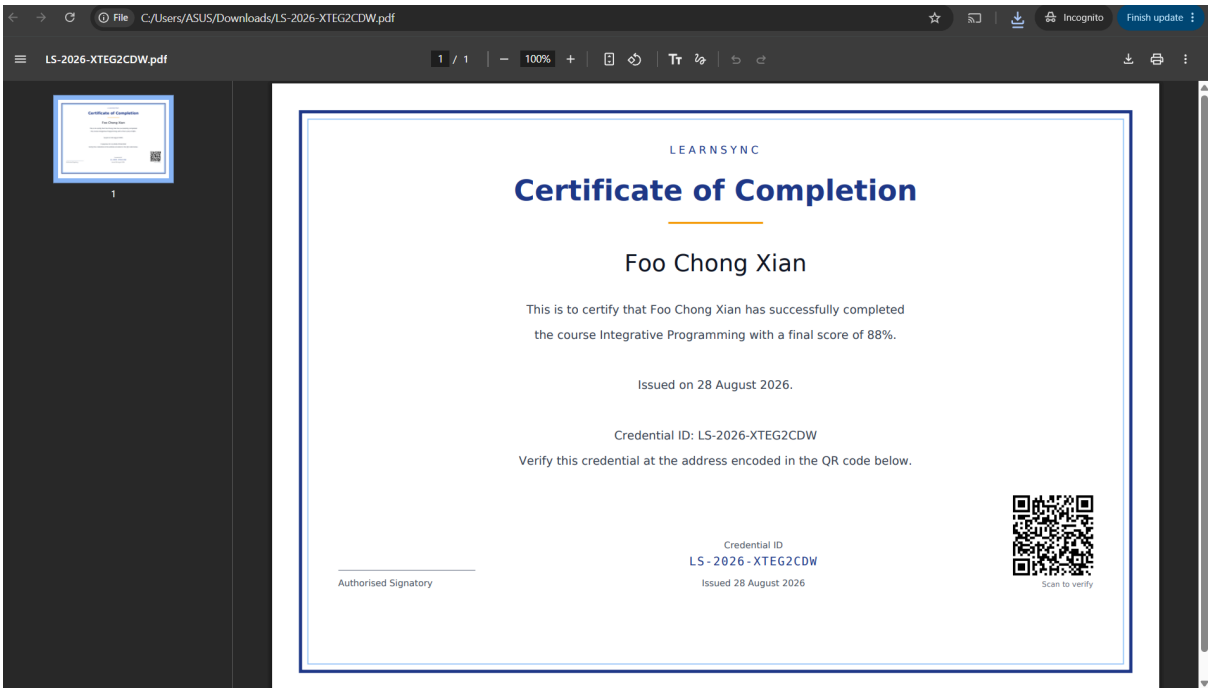



Figure 2.6: Generated Certificate PDF Displaying Credential ID and Verification QR Code

### F1.7: Public Credential Verification & Revocation

- Description: Anyone can visit /verify/{credential_id} without logging in. The page shows one of five answers: VALID, REVOKED, TAMPERED, EXPIRED or NOT FOUND. An admin can cancel a certificate and must give a reason. After that, the public page shows REVOKED straight away and the PDF can no longer be downloaded.
- Class Paths:
    - Controller: app/Http/Controllers/CertificateController.php (verify, revoke, adminIndex)
    - Subsystem: app/Patterns/Facade/Subsystem/IntegrityHasher.php (matches)
    - View Template: resources/views/certificates/verify.blade.php

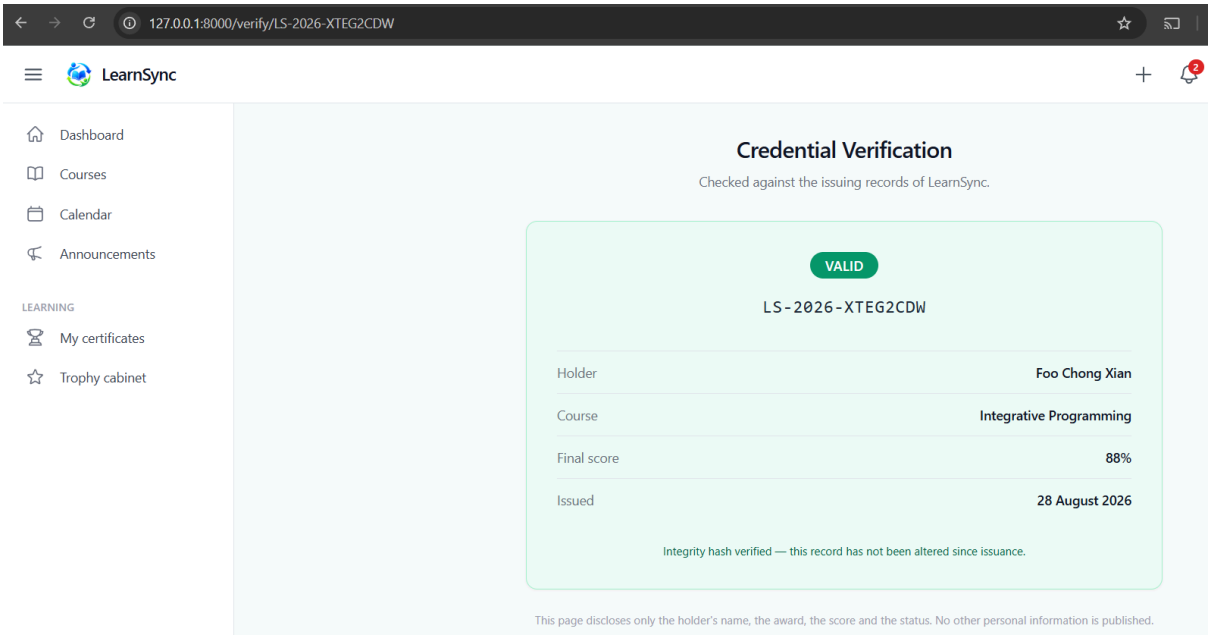



Figure 2.7: Public Verification Page Reporting VALID Status in a Logged-Out Browser

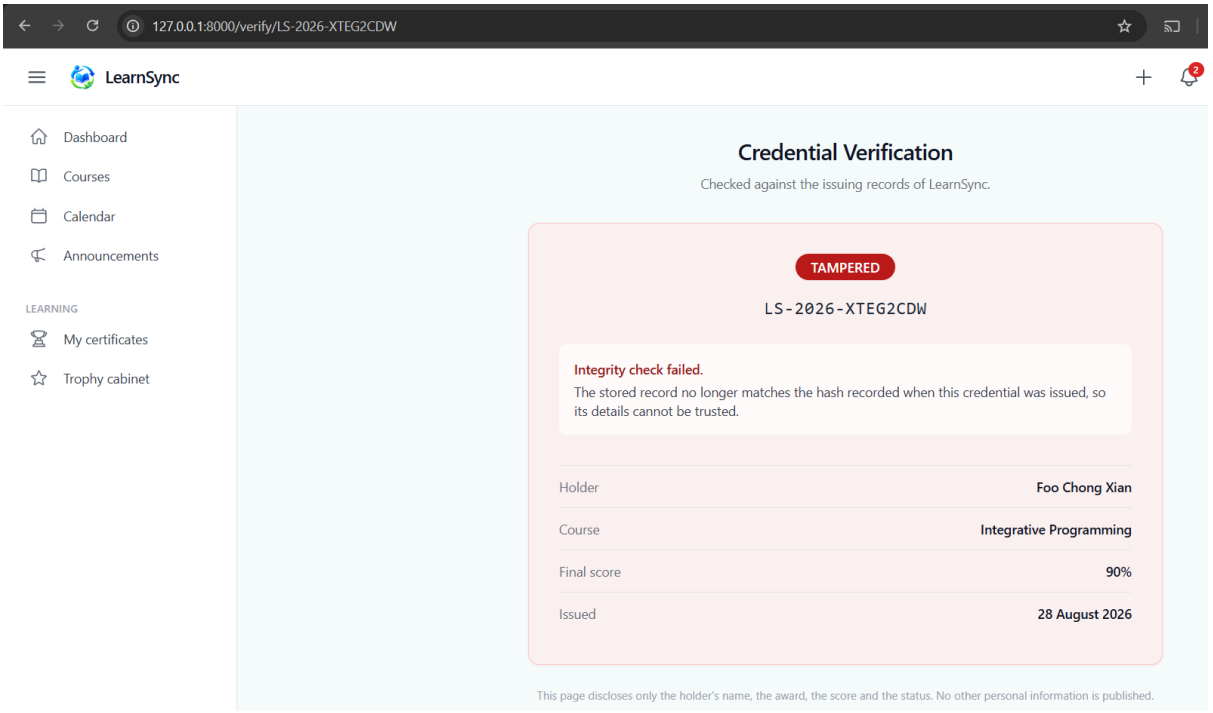



Figure 2.8: Same Credential Reporting TAMPERED After Direct Database Alteration of the Score

### F1.8: Administrator-Configurable Award Rules Engine

- Description: Badges and certificate rules are saved as data, not written into the code. Each rule has a name, a description, an award type of either badge or certificate, a tier, an icon or certificate design, a condition, a number, an optional subject, and an on or off switch. There are nine conditions to choose from. This is deliberately not a place where admins write code. They simply pick a condition from a list and type a number. Because of this, no rule an admin creates can crash the system, get stuck in a loop, or read data it should not see.

- Class Paths:
    - Controller: app/Http/Controllers/BadgeController.php (index, store, toggle, cabinet)
    - Subsystem: app/Patterns/Facade/Subsystem/AwardConditionEvaluator.php (isSatisfied)
    - Subsystem: app/Patterns/Facade/Subsystem/BadgeRuleEvaluator.php (evaluate)
    - View Templates: resources/views/badges/index.blade.php, resources/views/badges/cabinet.blade.php

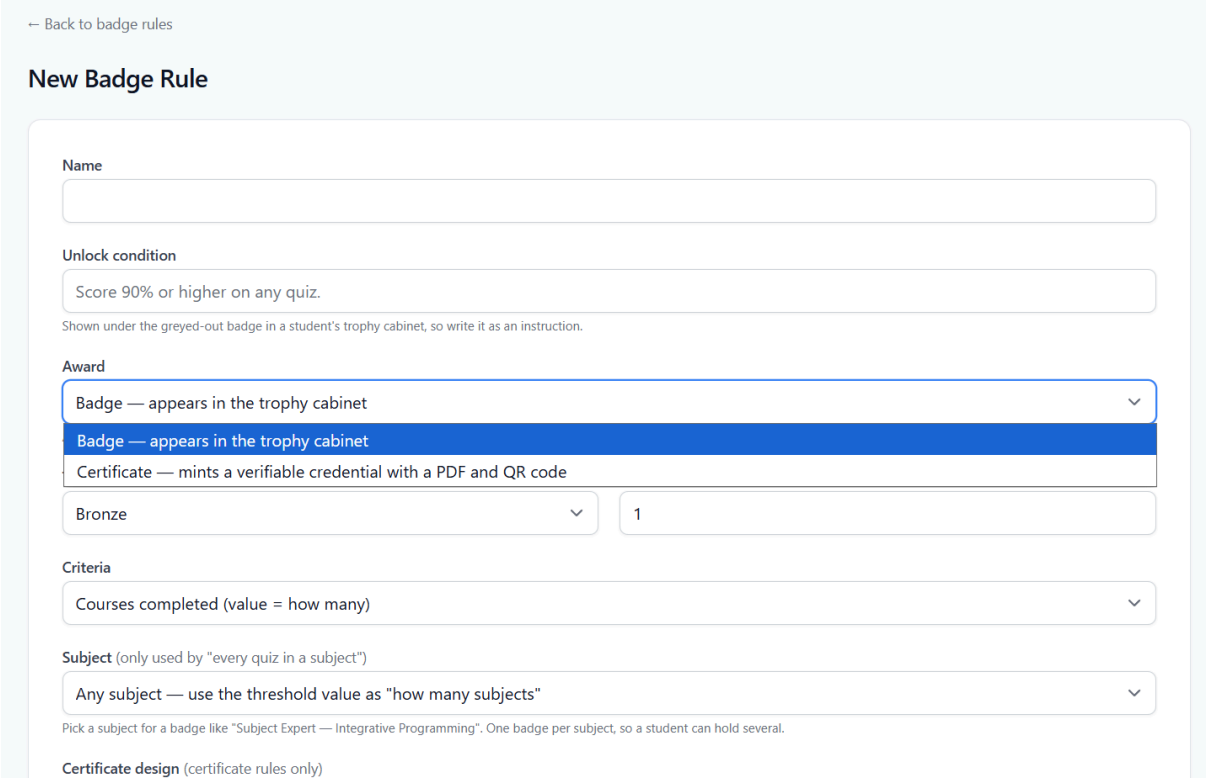



Figure 2.9: Administrator Award Rules Configuration Screen

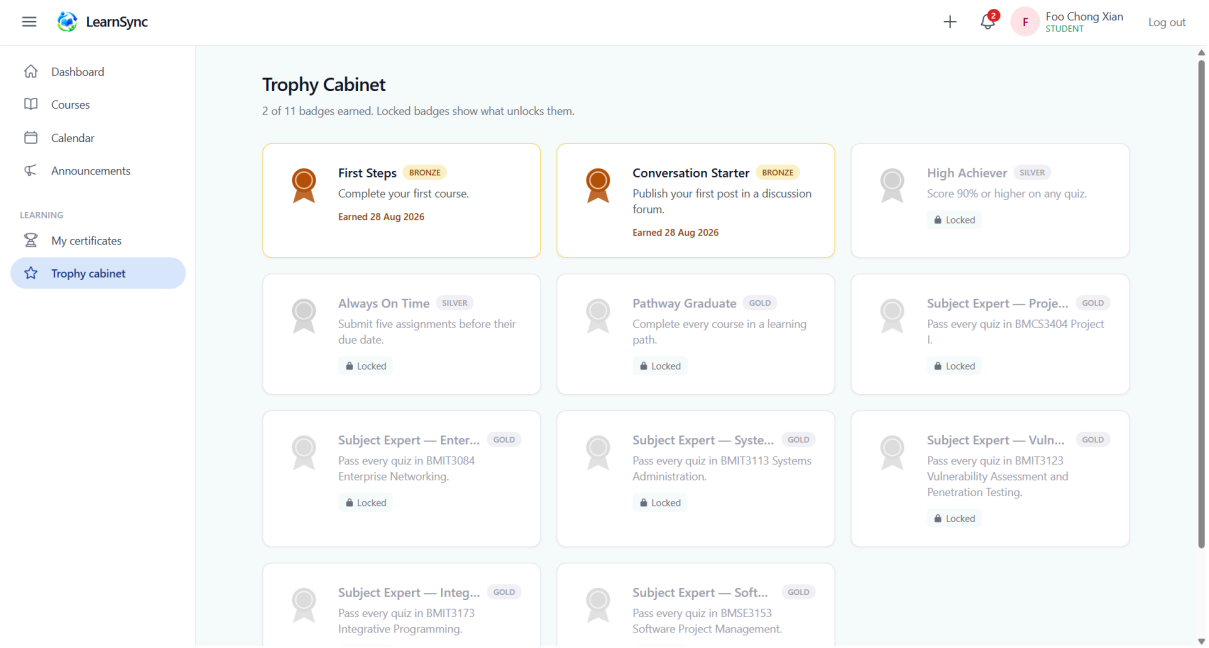
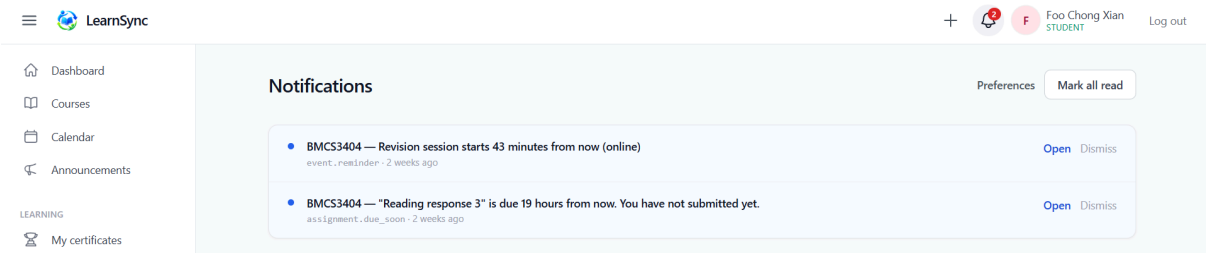



Figure 2.10: Student Trophy Cabinet Rendering Earned and Locked Badges

### F1.9: Notification Inbox & Per-User Preferences

- Description: Module 1 owns the notification inbox. This includes the bell icon with its unread count, the history page, the mark as read buttons, and the settings where each user can switch notification types on or off. The system checks these settings before saving a notification. So turning a type off stops it being created at all, rather than just hiding it.
- Class Paths:
    - Controller: app/Http/Controllers/NotificationController.php (index, readAll, editPreferences, updatePreferences)
    - Models: app/Models/Notification.php, app/Models/NotificationPreference.php
    - View Templates: resources/views/notifications/index.blade.php, resources/views/notifications/preferences.blade.php



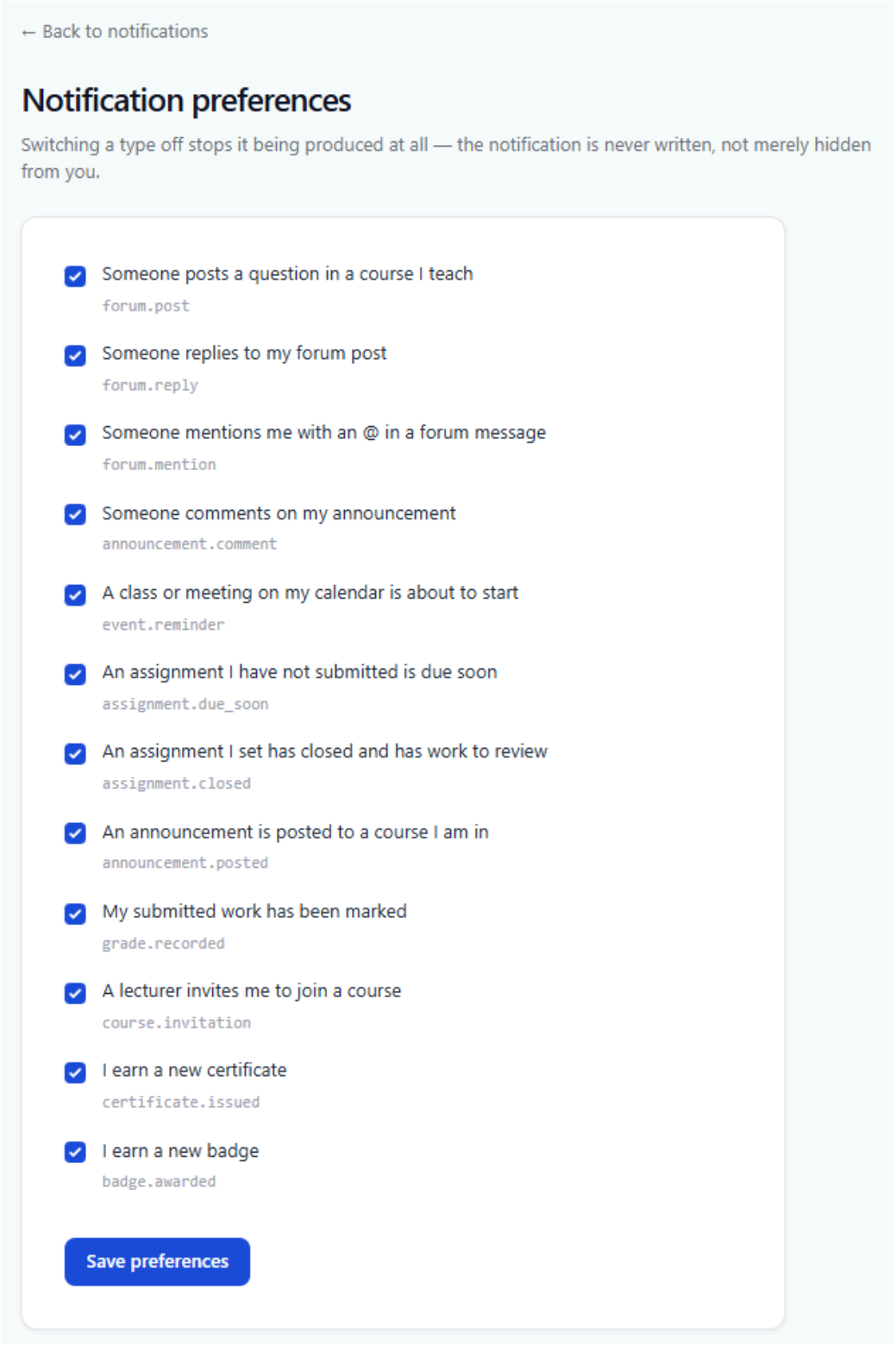



Figure 2.11: Notification Bell with Unread Badge and Preferences Configuration Screen

# 3. Entity Classes

## 3.1 Entity Class Diagram

The diagram below shows the entity classes using object references, meaning one object points to another object, instead of database foreign keys.

Diagram link:https://drive.google.com/file/d/1mDEfsh6UcQOtPgm46u2QRSk0cFGo9MRk/view?usp=sharing

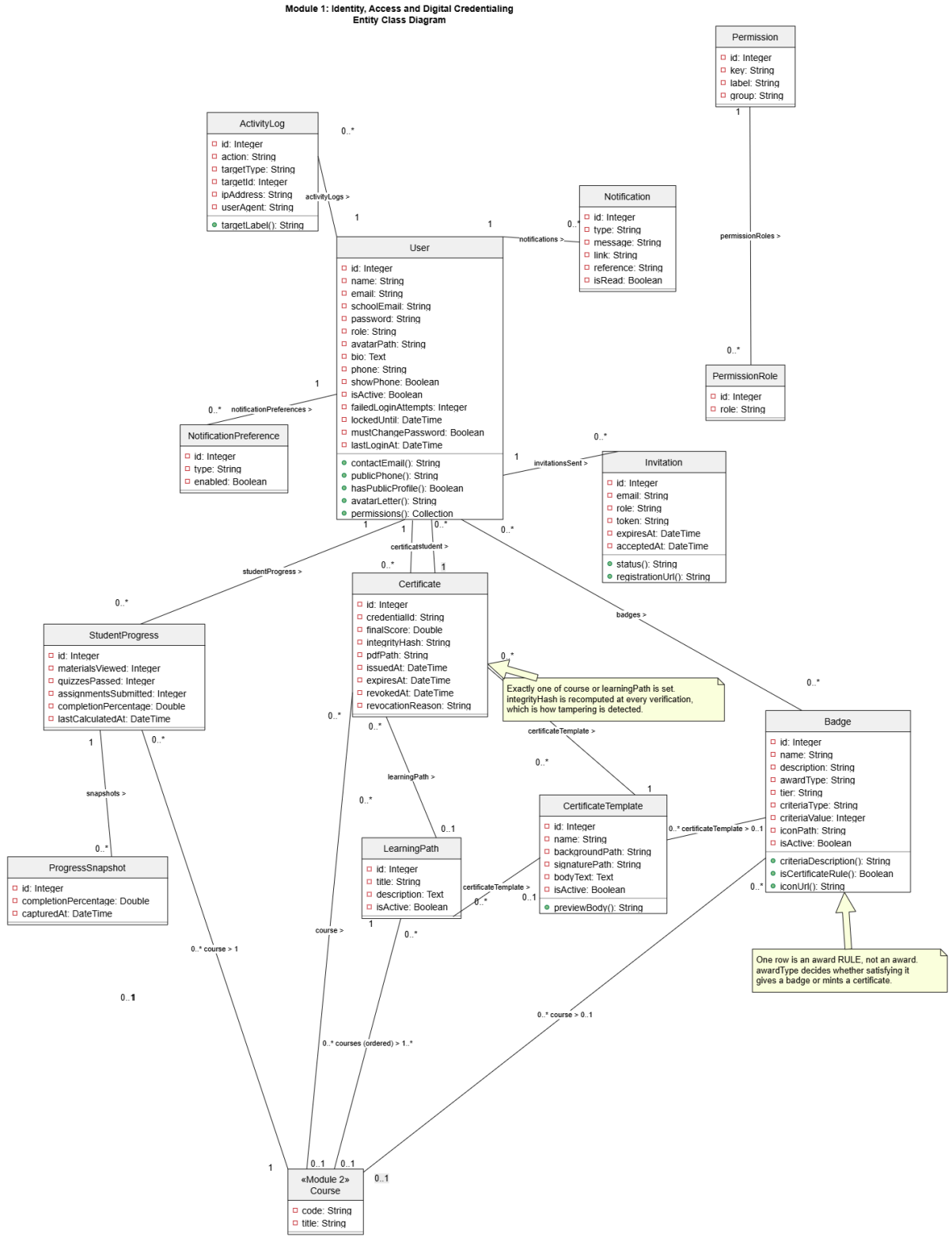



Figure 3.1: Entity Class Diagram for Module 1 Identity, Access & Digital Credentialing

### 3.2 Entity Class Implementation (Eloquent ORM Mapping)

The classes are written in PHP using Laravel's Eloquent ORM. Relationships are written as methods such as belongsTo, hasMany and belongsToMany, so the code never has to write SQL. For example, $certificate->student gives back a whole User object. So $certificate->student->name reads the student's name by following the link between objects, not by joining tables.

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Certificate extends Model
{
    protected $fillable = [
        'student_id', 'course_id', 'learning_path_id',
        'certificate_template_id', 'credential_id', 'final_score',
        'integrity_hash', 'pdf_path', 'issued_at', 'expires_at',
        'revoked_at', 'revocation_reason',
    ];

    protected function casts(): array
    {
        return [
            'issued_at'  => 'datetime',
            'expires_at' => 'datetime',
            'revoked_at' => 'datetime',
            'final_score' => 'double',
        ];
    }

   /** Object reference: A certificate belongs to a holder User */
    public function student(): BelongsTo
    {
        return $this->belongsTo(User::class, 'student_id');
    }

    /** Object reference: A certificate may attest to a Course */
    public function course(): BelongsTo
    {
        return $this->belongsTo(Course::class);
    }

    /** Object reference: A certificate is rendered from a CertificateTemplate */
    public function certificateTemplate(): BelongsTo
    {
        return $this->belongsTo(CertificateTemplate::class);
    }

    /** Domain behaviour expressed on the entity, not in a controller */
    public function isRevoked(): bool
```

```
    {
        return $this->revoked_at !== null;
    }
}
```

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;

class StudentProgress extends Model
{
    protected $fillable = [
        'student_id', 'course_id', 'materials_viewed', 'quizzes_passed',
        'assignments_submitted', 'completion_percentage', 'last_calculated_at',
    ];

    /** Object reference: Progress belongs to one student */
    public function student(): BelongsTo
    {
        return $this->belongsTo(User::class, 'student_id');
    }

    /** Object reference: Progress is measured against one Course */
    public function course(): BelongsTo
    {
        return $this->belongsTo(Course::class);
    }

    /** Object reference: Progress accumulates many historical snapshots */
    public function snapshots(): HasMany
    {
        return $this->hasMany(ProgressSnapshot::class);
    }
}
```

# 4. Design Pattern

## 4.1 Description of Design Pattern

For Module 1, I used the Facade Design Pattern, which is a Gang of Four Structural Pattern.

**What the pattern means:**

Gamma, Helm, Johnson and Vlissides (1994) describe the Facade Pattern as one that gives a single, unified way into a group of interfaces in a subsystem. The Facade sits at a higher level than those interfaces and makes the whole subsystem easier to use.

In simple words, some jobs need many different classes working together in the right order. Without a Facade, every part of the program that wants that job done must know all those classes, how to create them, and what order to call them in. That is a lot to remember. It also means that if the classes ever change, every caller must be fixed as well.

A Facade is one class that stands in front of all the others. It offers a few simple methods that say what you want, not how it is done. The caller only talks to the Facade.

**How this works in my module:**

- Facade: CredentialAuthority. It offers three simple methods, which are issueCertificate(), revoke() and verify().
- Subsystem: five helper classes in App\Patterns\Facade\Subsystem that do the real work. They are CredentialIdGenerator, IntegrityHasher, CertificateRenderer, ProgressCalculator and BadgeRuleEvaluator. The last one passes condition checks on to AwardConditionEvaluator.
- Callers: CertificateController and the Grade::created event. They only know the Facade. They never import DomPDF, the QR code maker, the settings table or the badge rules.

**How Facade is different from two similar patterns:**

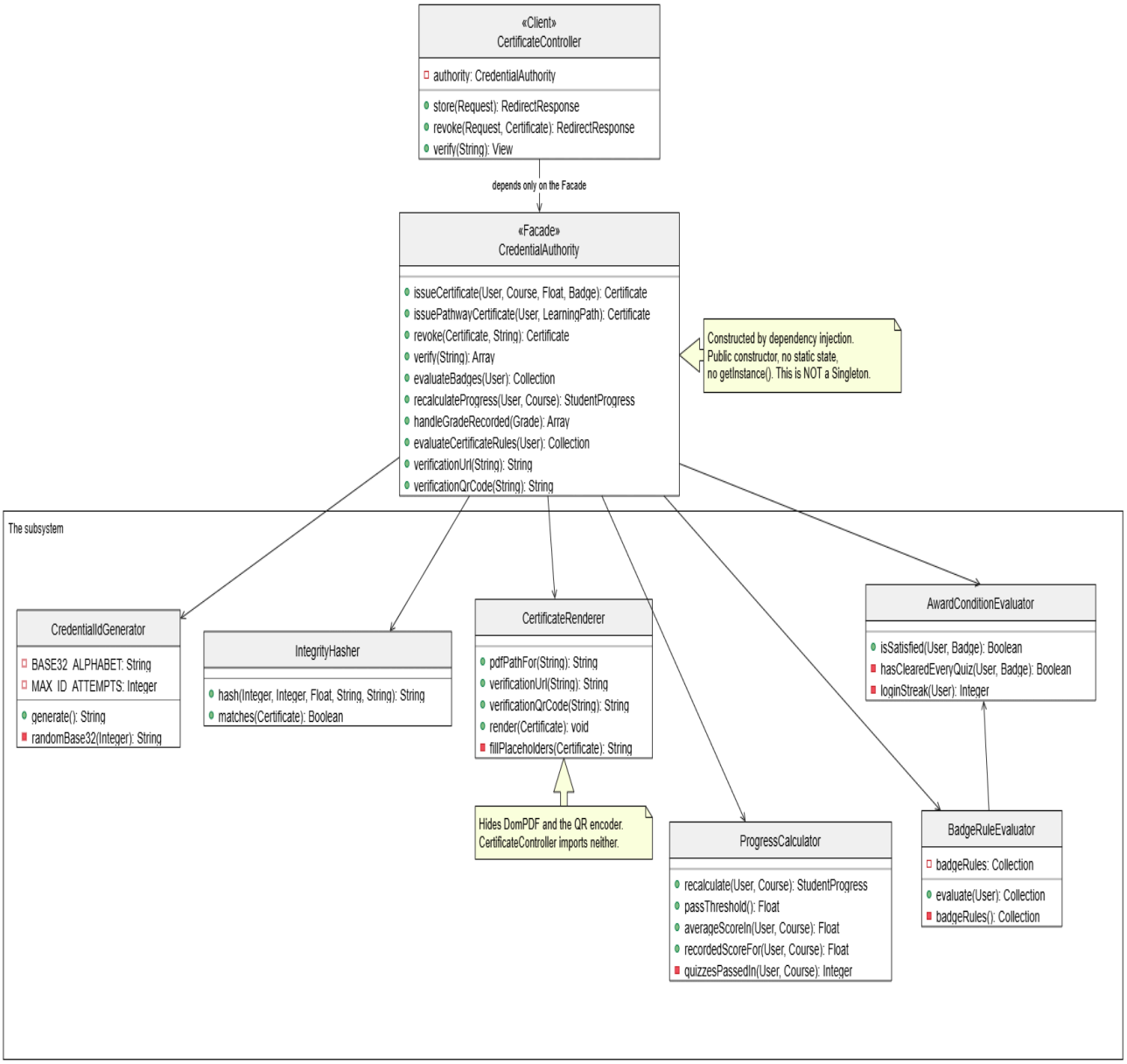
A Facade does not block the subsystem. The five helper classes can still be used directly and tested on their own. The Facade only adds an easier way in. This is different from Proxy, which controls access to one object.

A Facade also creates a new and simpler interface. This is different from Adapter, which Module 2 uses. Adapter changes one interface into another shape without making it simpler. Adapter makes two different things look the same, while Facade makes many things look like one.

Diagram link:https://drive.google.com/file/d/1qQWBn3jtyRRLgKdJ-_6wiHOV65SNLlG4/view?usp=sharing

Figure 4.1: Facade Pattern Class Diagram Showing CredentialAuthority and Its Subsystem

**Module 1: Facade Pattern**
**CredentialAuthority and its subsystem**



## 4.2 Implementation of Design Pattern

### 1. Helper Class for Making the Certificate ID

**(app/Patterns/Facade/Subsystem/CredentialIdGenerator.php)**

```
namespace App\Patterns\Facade\Subsystem;

use App\Models\Certificate;
use RuntimeException;

class CredentialIdGenerator
{
    private const CREDENTIAL_PREFIX = 'LS';
    private const CREDENTIAL_RANDOM_LENGTH = 8;

    // The letters I, L, O and U are missing on purpose, so a printed ID
    // can never be misread. 1 is never confused with I or L, and 0 is
    // never confused with O.
    private const BASE32_ALPHABET = '0123456789ABCDEFGHJKMNPQRSTVWXYZ';
    private const MAX_ID_ATTEMPTS = 10;

    public function generate(): string
    {
        for ($attempt = 0; $attempt < self::MAX_ID_ATTEMPTS; $attempt++) {
            $candidate = sprintf('%s-%s-%s',
                self::CREDENTIAL_PREFIX, now()->year,
                $this->randomBase32(self::CREDENTIAL_RANDOM_LENGTH));

            // Try again if this ID somehow already exists
            if (! Certificate::where('credential_id', $candidate)->exists()) {
                return $candidate;
            }
        }

        throw new RuntimeException('Could not make a unique credential ID.');
    }

    private function randomBase32(int $length): string
    {
        $lastIndex = strlen(self::BASE32_ALPHABET) - 1;
        $output = '';

        for ($i = 0; $i < $length; $i++) {
            // random_int() is the secure random function.
            // rand() and mt_rand() can be guessed, so they are not used.
            $output .= self::BASE32_ALPHABET[random_int(0, $lastIndex)];
        }

        return $output;
    }
}
```

**2. Helper Class for the Security Code**

**(app/Patterns/Facade/Subsystem/IntegrityHasher.php)**

```
namespace App\Patterns\Facade\Subsystem;

use App\Models\Certificate;

class IntegrityHasher
{
    // Joins the important details together and turns them into one code
    public function hash(int $studentId, ?int $courseId, float $score,
                         string $issuedAt, string $credentialId): string
    {
        return hash('sha256', implode('|', [
            $studentId, $courseId ?? '', $score, $issuedAt, $credentialId,
        ]));
    }

    public function matches(Certificate $certificate): bool
    {
        // Work the code out again from the record as it is right now
        $recomputed = $this->hash(
            $certificate->student_id,
            $certificate->course_id,
            $certificate->final_score,
            $certificate->issued_at->format('Y-m-d H:i:s'),
            $certificate->credential_id
        );

        // hash_equals() always takes the same amount of time to compare,
        // so an attacker cannot guess the code by timing the check
        return hash_equals($certificate->integrity_hash, $recomputed);
    }
}
```

**3. The Facade (app/Patterns/Facade/CredentialAuthority.php)**

```
namespace App\Patterns\Facade;

use App\Models\Certificate;
use App\Models\CertificateTemplate;
use App\Models\Course;
use App\Models\User;
use App\Patterns\Facade\Subsystem\AwardConditionEvaluator;
use App\Patterns\Facade\Subsystem\BadgeRuleEvaluator;
use App\Patterns\Facade\Subsystem\CertificateRenderer;
use App\Patterns\Facade\Subsystem\CredentialIdGenerator;
use App\Patterns\Facade\Subsystem\IntegrityHasher;
use App\Patterns\Facade\Subsystem\ProgressCalculator;
use Illuminate\Support\Facades\DB;

class CredentialAuthority
{
    // Laravel gives the Facade its five helper classes automatically
    public function __construct(
        private CredentialIdGenerator $ids,
        private IntegrityHasher $hasher,
        private CertificateRenderer $renderer,
        private ProgressCalculator $progress,
        private BadgeRuleEvaluator $badges,
        private AwardConditionEvaluator $conditions,
    ) {
    }

    public function issueCertificate(User $student, Course $course,
                                     ?float $finalScore = null,
                                     ?Badge $rule = null): Certificate
    {
        $template = $rule?->certificateTemplate
            ?? CertificateTemplate::where('is_active', true)->first();

        $score = $finalScore ?? $this->progress->recordedScoreFor($student, $course);

        // Everything inside runs together. If any step fails, all steps
        // are undone, so a half made certificate can never exist.
        return DB::transaction(function () use ($student, $course, $template, $score) {

            $credentialId = $this->ids->generate();          // helper 1
            $issuedAt = now();

            $certificate = Certificate::create([
                'student_id'    => $student->id,
                'course_id'     => $course->id,
                'certificate_template_id' => $template->id,
                'credential_id' => $credentialId,
                'final_score'   => $score,
                'integrity_hash' => $this->hasher->hash(     // helper 2
                    $student->id, $course->id, $score,
```

```
                $issuedAt->format('Y-m-d H:i:s'), $credentialId
            ),
            'pdf_path'      => $this->renderer->pdfPathFor($credentialId),
            'issued_at'     => $issuedAt,
        ]);

        $this->renderer->render($certificate);          // helper 3
        ActivityLog::record('certificate.issued', $certificate);
        $this->issueCompletedPathways($student, $course);
        $this->evaluateBadges($student);                // helper 5

        return $certificate;
    });
  }

  public function verify(string $credentialId): array
  {
    $certificate = Certificate::with(['student', 'course'])
        ->where('credential_id', $credentialId)->first();

    if ($certificate === null) {
        return ['status' => 'not_found', 'certificate' => null];
    }

    if ($certificate->revoked_at !== null) {
        return ['status' => 'revoked', 'certificate' => $certificate];
    }

    // helper 2 checks whether anyone changed the record
    if (! $this->hasher->matches($certificate)) {
        return ['status' => 'tampered', 'certificate' => $certificate];
    }

    return ['status' => 'valid', 'certificate' => $certificate];
  }
}
```

**4. How the Controller Uses It (app/Http/Controllers/CertificateController.php)**

```
class CertificateController extends Controller
{
    // Laravel passes in the Facade. The controller knows nothing else.
    public function __construct(private CredentialAuthority $authority)
    {
    }

    public function store(Request $request): RedirectResponse
    {
        abort_unless($request->user()->can('certificate.issue'), 403);

        // Eleven separate jobs, done with one line
        $certificate = $this->authority->issueCertificate(
            $student, $course, (float) $data['final_score']
        );

        return redirect()->route('admin.certificates.index')
            ->with('success', "Issued credential {$certificate->credential_id}.");
    }
}
```

**5. Registering the Facade (app/Providers/CredentialServiceProvider.php)**

```
class CredentialServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        // One instance per web request. This is just Laravel managing how
        // long an object lives. It is NOT the Singleton pattern, because
        // the constructor is public, there is no static variable, and
        // there is no getInstance() method anywhere.
        $this->app->scoped(CredentialAuthority::class);
    }
}
```

### 4.3 Justification of Design Pattern

1. The job really is complicated. Making one certificate takes eleven steps. The system must create a unique ID, make the security code, fill in the template wording, make the PDF, make the QR code, save the file, recalculate progress, save a progress snapshot, check every award rule, check for a finished learning path, and write the security log. That needs five helper classes and four outside libraries. Without the Facade, CertificateController would have to do all of this itself. A controller is not supposed to know about PDF libraries and QR codes, and doing so would break the Single Responsibility Principle.

2. The outside stays the same when the inside changes. During development I rewrote the PDF part, the badge engine and the progress calculation. I also added a whole sixth helper class called AwardConditionEvaluator when award rules became configurable by admins. Not a single caller had to change, because issueCertificate(), revoke() and verify() kept the same names and parameters. This follows the Open Closed Principle, and the project's own commit history proves it happened.

3. The helper classes can still be tested on their own. A Facade does not lock the subsystem away. Each of the five helpers can be created and tested by itself. This is important, because the old Singleton did the opposite. It made a second instance impossible, which stopped me testing pieces separately and gave nothing useful in return.

4. The old reason for using Singleton was actually wrong. The Singleton was justified by saying two copies running at once could create the same certificate ID twice. In PHP this is not true. Every web request runs in its own separate process with its own memory, so two requests always had two separate objects anyway. The Singleton never protected anything across requests. What really stops duplicate IDs is the unique index on the certificates.credential_id column, together with the retry loop inside CredentialIdGenerator. I did not change either of them. So moving to Facade lost nothing and gave the module a reason that actually holds up.

# 5. Software Security

## 5.1 Potential Threat/Attack

**Threat 1: Fake or Edited Certificates (OWASP A08: Software and Data Integrity Failures)**

**Attack Description:**

The public checking page is the whole point of the certificate feature, which makes it the biggest target in the system. It can be attacked in two ways.

The first way is changing the record directly. Certificates are just rows in a MySQL database. Someone who can reach the database can run one simple UPDATE command and change a mark from 45 to 95. They might reach it through a stolen phpMyAdmin login, leaked database passwords, an SQL injection hole somewhere else in the app, or by being a dishonest insider such as a student helper with database access. No web request is involved at all, so checking form input cannot stop it.

The second way is making up a certificate ID. If IDs were counted in order, such as LS-2026-00001 and then LS-2026-00002, an attacker could try them one by one to view other people's certificates. They could also invent a real looking ID, print it on a fake paper certificate, and hope the employer reads the number instead of scanning the QR code.

**Risk Impact:**

The college would be publicly confirming a fake qualification through its own website. This destroys the trust the whole system depends on, and could help someone get a job using marks they never earned.

**Threat 2: Password Guessing Attacks on Login (OWASP A07: Identification and Authentication Failures)**

**Attack Description:**

Module 1 owns the login page, which protects every other module, and it is open to the whole internet.

In a brute force attack, a program tries password after password on one account. College email addresses follow an obvious pattern, so the attacker already knows real usernames. With no limit on attempts, thousands of guesses can be tried, and a weak password will be found quite quickly.

In a credential stuffing attack, which works far better in real life, the attacker uses email and password pairs stolen from a completely different website that was hacked before. Many people reuse the same password everywhere. Because each pair is only tried once per account, simple attempt limits are often never noticed.

**Risk Impact:**

How bad this is depends on whose account is taken. A stolen student account lets someone hand in work and read grades. A stolen lecturer account lets someone change marks, and because marks automatically trigger certificates, this can create fake certificates. A stolen admin account lets someone change permissions and issue or cancel certificates, which means the whole system's trust is gone.

## 5.2 Secure Coding Practice

### Secure Practice 1: A Security Code That Detects Any Change

OWASP Category: Cryptographic Practices. Instead of trusting the saved record, every certificate carries a security code that is worked out again every single time someone checks it.

```
// app/Patterns/Facade/Subsystem/IntegrityHasher.php
public function matches(Certificate $certificate): bool
{
    // SECURITY (Module 1): work the code out again from the record AS IT IS NOW
    $recomputed = $this->hash(
        $certificate->student_id,
        $certificate->course_id,
        $certificate->final_score,
        $certificate->issued_at->format('Y-m-d H:i:s'),
        $certificate->credential_id
    );

    // SECURITY (Module 1): use hash_equals(), never ===
    return hash_equals($certificate->integrity_hash, $recomputed);
}
```

How this stops the attack: if someone changes the mark in the database, the ingredients that make the code have changed too. The newly worked out code no longer matches the saved one, so the page shows TAMPERED instead of VALID. To fake it successfully, the attacker would need to find different data that produces the exact same SHA-256 code, which is not possible with today's computers.

I used hash_equals() instead of === on purpose. hash_equals() always takes the same amount of time, no matter how much of the code matches. A normal === comparison stops early at the first wrong letter, and an attacker could measure those tiny time differences to work out the correct code one letter at a time.

Fake IDs are stopped in a different way, by making IDs random instead of counted.

```
// app/Patterns/Facade/Subsystem/CredentialIdGenerator.php
private function randomBase32(int $length): string
{
    $lastIndex = strlen(self::BASE32_ALPHABET) - 1;
    $output = '';

    for ($i = 0; $i < $length; $i++) {
        // SECURITY (Module 1): random_int() uses the operating system's
        // secure random source. rand() and mt_rand() can be predicted
        // after watching enough output, so they must not be used here.
        $output .= self::BASE32_ALPHABET[random_int(0, $lastIndex)];
    }
```

```
    return $output;
}
```

With 32 possible characters in 8 positions, there are about 1.1 trillion possible IDs. Trying them one by one is not realistic, and the IDs actually issued are only a tiny fraction of that.

**Secure Practice 2: Locking Accounts, Blocking Old Passwords and Logging Every Attempt**

OWASP Category: Authentication and Password Management. There are four layers here, and none of them depends on the user choosing a good password.

```
// app/Providers/AppServiceProvider.php
private function registerAuditListeners(): void
{
    // SECURITY (Module 1): count every failed login on a real account.
    // This listens to Laravel's own event, so any new login method
    // added later is protected automatically.
    Event::listen(Failed::class, function (Failed $event) {
        if ($event->user === null) {
            return;   // do not reveal which email addresses exist
        }

        ActivityLog::record('auth.failed', null, $event->user);
        $event->user->increment('failed_login_attempts');
    });

    // SECURITY (Module 1): a good login resets the counter, so a real
    // user who mistypes twice is never punished for it.
    Event::listen(Login::class, function (Login $event) {
        ActivityLog::record('auth.login', null, $event->user);

        $event->user->forceFill([
            'last_login_at' => now(),
            'failed_login_attempts' => 0,
        ])->save();
    });
}
```

(a) Locking the account. After five wrong passwords in a row the account locks, and only an admin can unlock it. There is no option to try again in fifteen minutes. This is deliberate. A timed unlock only slows an automatic attack down, but a real lock stops it completely and makes a human look at what happened.

(b) Blocking old passwords. A new password is compared against the last three saved passwords and refused if it matches. This stops a user going back to an old password that might already be in a leaked password list, which is exactly what a credential stuffing attack uses.

(c) Bcrypt password storage. Passwords are saved using bcrypt, never as plain text. Bcrypt is slow on purpose and adds a different random salt to every password. So even if someone steals the whole database, cracking the passwords is very slow, and pre made lookup tables are useless.

(d) Logging every login attempt. Every login, logout and failed attempt is saved with the IP address and browser. This is the layer that lets you notice an attack. A credential stuffing

attack looks like lots of auth.failed rows from just a few IP addresses across many different accounts. Without logging, that pattern is completely invisible. An admin can filter and export these records to look at them.

## 6. Web Services

Module 1 works with web services in both directions.

As a provider, it offers the certificate checking service. This was the obvious choice, because checking a certificate is already a public, read only lookup that returns a fixed set of fields. It is also the one thing other modules and outside people actually need from Module 1.

As a consumer, it calls Module 2's course information service. When a certificate is printed, the course code and title come from Module 2, because Module 2 owns all course data. Module 1 never reads Module 2's tables directly.

I chose REST with JSON instead of SOAP because it is lighter, needs no WSDL contract file or XML wrapper, and can be used straight away by a browser, by Postman, and by PHP.

### 6.1 Web Service Exposure

**Interface Agreement (IFA) for Service Exposure**

| | **Description** |
|---|---|
| Protocol | RESTful Web Service (JSON over HTTP) |
| Function Description | *Returns the status and public details of a certificate, looked up by its credential ID* |
| Source Module | Module 1: Identity, Access and Digital Credentialing Module |
| Target Module | Module 5 (Academic Progress Analytics), Module 2 (Academic Resources Repository), outside verifiers |
| HTTP Method & URL | GET /api/credentials/verify |
| Controller Action | App\Http\Controllers\Api\CredentialApiController@verify |
| Function Name | getCredentialStatus |

**Web Services Request Parameters (IFA Requirement)**

| **Field Name** | **Field Type** | **Mandatory/ Optional** | **Description** | **Format** |
|---|---|---|---|---|
| credentialId | Integer | Mandatory | The ID of the certificate to check. | LS-YYYY-XXXXXX XX<br>Letters and numbers only |
| detailFlag | Integer | Mandatory | How much detail is needed. | 1: get contact no.<br>2: get Address<br>3: get both contact number & address |
| requestID | String | Mandatory | A unique tracking ID made by Module 1. | YYYY-MM-DD HH:MM:SS |

| | | | | |
|---|---|---|---|---|
| timeStamp | String | Mandatory | The time the request was sent. | YYYY-MM-DDTHH:MM:SSZ |

**Web Services Response Parameters (IFA Requirement)**

| Field Name | Field Type | Mandatory/ Optional | Description | Format |
|---|---|---|---|---|
| status | String | Mandatory | Whether the request worked. | S: Success<br>F: Fail<br>E: Error |
| timeStamp | String | Mandatory | The time the answer was created. | YYYY-MM-DDTHH:MM:SSZ |
| data.courseCode | String | Mandatory | The public course code. | Letters and numbers, e.g. BMIT3173 |
| data.courseTitle | String | Mandatory | The full course name. | Letters and numbers |
| data.instructorName | String | Optional | The lecturer's name. | Only sent when queryFlag is 2 |

**Code Implementation (app/Http/Controllers/Api/CredentialApiController.php)**

```
namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Patterns\Facade\CredentialAuthority;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class CredentialApiController extends Controller
{
    // The Facade again. Adding a web service needed no knowledge of
    // hashing, PDFs or badge rules.
    public function __construct(private CredentialAuthority $authority)
    {
    }

    public function verify(Request $request): JsonResponse
    {
        try {
            // Check the four required IFA fields are present and correct
            $validator = validator($request->all(), [
                'credentialId' => ['required', 'regex:/^LS-\d{4}-[0-9A-Z]{8}$/'],
                'detailFlag'   => ['required', 'integer', 'in:1,2,3'],
                'requestID'    => ['required', 'string', 'max:64'],
                'timeStamp'    => ['required', 'date'],
            ]);

            if ($validator->fails()) {
                return $this->ifaResponse('F', [
                    'requestID' => $request->input('requestID'),
                    'credentialStatus' => 'NOT_FOUND',
                ], 400);
            }

            $result = $this->authority->verify($request->input('credentialId'));
            $certificate = $result['certificate'];
            $flag = (int) $request->input('detailFlag');

            $data = [
                'requestID' => $request->input('requestID'),
                'credentialStatus' => strtoupper($result['status']),
            ];

            // Only send the holder's details if the caller asked for them
            if ($certificate !== null && $flag >= 2) {
                $data['holderName']  = $certificate->student->name;
                $data['courseTitle'] = $certificate->course?->title;
                $data['finalScore']  = round($certificate->final_score, 2);
                $data['issuedDate']  = $certificate->issued_at->format('Y-m-d');
            }

            if ($certificate !== null && $flag === 3) {
                $data['credentialDetails'] = [
```

```
                'issuer'           => config('app.name'),
                'expiresAt'        => $certificate->expires_at?->format('Y-m-d'),
                'revocationReason' => $certificate->revocation_reason,
            ];
        }

        return $this->ifaResponse('S', $data, 200);

    } catch (\Exception $e) {
        return $this->ifaResponse('E', [
            'message' => 'Internal server error: ' . $e->getMessage(),
        ], 500);
    }
  }

  // Every answer carries status and timeStamp, as the IFA requires
  private function ifaResponse(string $status, array $data, int $code): JsonResponse
  {
    return response()->json([
        'status'    => $status,
        'timeStamp' => now()->toIso8601String(),
        'data'      => $data,
    ], $code);
  }
}
```

```
// routes/api.php
use App\Http\Controllers\Api\CredentialApiController;
use Illuminate\Support\Facades\Route;

// Public on purpose, because checking a certificate must work with no account.
Route::get('/credentials/verify', [CredentialApiController::class, 'verify']);
```

```
{
  "status": "S",
  "timeStamp": "2026-08-28T14:30:02Z",
  "data": {
    "requestID": "REQ-CRED-84920",
    "credentialStatus": "VALID",
    "holderName": "Foo Chong Xian",
    "courseTitle": "Integrative Programming",
    "finalScore": 88.00,
    "issuedDate": "2026-08-28"
  }
}
```

## 6.2 Web Service Consumption

### Interface Agreement (IFA) for Service Consumption

| | Description |
|---|---|
| Protocol | RESTful Web Service (JSON over HTTP) |
| Function Description | Returns the verification status of a credential, looked up by its credential ID |
| Source Module | Module 1: Identity, Access and Digital Credentialing Module |
| Consuming Module | Module 5: Academic Progress Analytics and Evaluation Module |
| HTTP Method & URL | GET /api/credentials/verify |
| Client Class | App\Support\Api\CredentialStatusClient@status |
| Function Name | getCredentialStatus |
| Protocol | RESTful Web Service (JSON over HTTP) |

### Web Services Request Parameters (IFA Requirement)

| Field Name | Field Type | Mandatory/ Optional | Description | Format |
|---|---|---|---|---|
| credentialId | String | Mandatory | The credential to check. | LS-YYYY-XXXXXXXX |
| detailFlag | Integer | Mandatory | How much detail is wanted. | Module 5 always sends 1 |
| requestID | String | Mandatory | A tracking ID made by Module 5, prefixed CRED-REQ. | Letters, numbers, hyphens |
| timeStamp | String | Mandatory | The time the request was sent. | YYYY-MM-DDTHH:MM:SSZ |

**Web Services Response Parameters (IFA Requirement for Consumption)**

| Field Name | Field Type | Mandatory / Optional | Description | Format / Values |
|---|---|---|---|---|
| status | String | Mandatory | Whether the request worked. | S for Success, F for Fail, E for Error |
| timeStamp | String | Mandatory | The time the answer was created. | YYYY-MM-DDTHH:MM:SSZ |
| data.credentialStatus | String | Mandatory | The result of the check. | VALID, REVOKED, TAMPERED, EXPIRED, NOT_FOUND |
| data.holderName | String | Optional | Not requested by Module 5. | Only returned at detailFlag 2 or above |

**Consumption Code Implementation (app/Support/CourseInfoClient.php)**

```
namespace App\Support;

use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Log;

class CourseInfoClient
{
    public function fetch(int $courseId): ?array
    {
        try {
            // Call Module 2's course information web service
            $response = Http::timeout(10)->acceptJson()->get(
                config('app.url') . '/api/courses/info',
                [
                    'courseId'  => $courseId,
                    'queryFlag' => 1,
                    'requestID' => uniqid('CRS-REQ-'),
                    'timeStamp' => now()->toIso8601String(),
                ]
            );

            if ($response->successful()) {
                $payload = $response->json();

                // Always check the IFA status field before trusting the data
                if (($payload['status'] ?? null) === 'S') {
                    return [
                        'code'  => $payload['data']['courseCode'],
                        'title' => $payload['data']['courseTitle'],
                    ];
                }
            }

            return null;

        } catch (\Exception $e) {
            Log::error('Module 1 could not get course info from Module 2',
                    ['error' => $e->getMessage()]);

            // If Module 2 is down, we still issue the certificate using
            // our own copy of the title. A failed lookup must never stop
            // a student getting the certificate they earned.
            return null;
        }
    }
}
```

## 7. References

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.

Laravel LLC. (2026). Laravel 12.x documentation: Service container, Eloquent ORM and authentication. https://laravel.com/docs/12.x

OWASP Foundation. (2021). OWASP Top 10:2021 The ten most critical web application security risks. Open Web Application Security Project. https://owasp.org/Top10/

OWASP Foundation. (2022). OWASP secure coding practices quick reference guide (Version 2.1). https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/

PHP Group. (2026). PHP manual: hash_equals and random_int. https://www.php.net/manual/en/function.hash-equals.php

United Nations. (2015). Transforming our world: The 2030 agenda for sustainable development (Goal 4: Quality Education). United Nations Department of Economic and Social Affairs. https://sdgs.un.org/goals/goal4

# 8. Appendices

## Appendix A: Automated Testing Results

```
c:\xampp\htdocs\edusystem>php artisan test

  PASS  Tests\Unit\ExampleTest
  ✓ that true is true                                                    0.01s

  PASS  Tests\Feature\Auth\AuthenticationTest
  ✓ login screen can be rendered                                         0.59s
  ✓ users can authenticate using the login screen                        0.08s
  ✓ users can not authenticate with invalid password                     0.24s
  ✓ users can logout                                                     0.03s

  PASS  Tests\Feature\Auth\EmailVerificationTest
  ✓ email verification screen can be rendered                            0.04s
  ✓ email can be verified                                                0.04s
  ✓ email is not verified with invalid hash                              0.03s

  PASS  Tests\Feature\Auth\PasswordConfirmationTest
  ✓ confirm password screen can be rendered                              0.04s
  ✓ password can be confirmed                                            0.03s
  ✓ password is not confirmed with invalid password                      0.23s

  PASS  Tests\Feature\Auth\PasswordResetTest
  ✓ reset password link screen can be rendered                           0.03s
  ✓ reset password link can be requested                                 0.25s
  ✓ reset password screen can be rendered                                0.26s
  ✓ password can be reset with valid token                               0.28s

  PASS  Tests\Feature\Auth\PasswordUpdateTest
  ✓ password can be updated                                              0.04s
  ✓ correct password must be provided to update password                 0.03s

  PASS  Tests\Feature\Auth\RegistrationTest
  ✓ open registration route does not exist                               0.04s
  ✓ an unknown token is rejected                                         0.03s
  ✓ an expired invitation is refused                                     1.66s
  ✓ an already accepted invitation is refused                            0.56s
  ✓ a valid invitation creates the account with its fixed role           0.05s
  ✓ a token cannot be redeemed twice                                     0.60s

  PASS  Tests\Feature\AwardAndActivityNotificationTest
  ✓ earning a certificate notifies the holder                            0.53s
  ✓ earning a badge notifies the student                                 0.05s
  ✓ marking submitted work notifies the student                          0.04s
  ✓ a marked quiz does not notify because the result is already on screen 0.03s
```

```
✓ inviting a student to a course notifies them                                  0.03s
✓ a switched off preference still suppresses the new types                      0.03s

 PASS  Tests\Feature\AwardRuleTest
✓ an admin defined average score rule awards a badge                            0.06s
✓ an admin defined quizzes completed rule counts distinct quizzes               0.05s
✓ an admin defined certificate rule mints a real credential                     0.41s
✓ a certificate rule does not mint twice                                        0.56s
✓ a certificate rule is never handed out as a badge                             0.50s
✓ deactivating a rule stops it without removing awards already made             0.06s

 PASS  Tests\Feature\CalendarEventDetailTest
✓ an enrolled student can open an events detail page                            0.07s
✓ a student cannot open an event for a course they are not in                   0.04s
✓ a lecturer cannot open an event for another lecturers course                  0.04s
✓ an institution wide event is visible to everyone                              0.05s
✓ a meeting shows a join button                                                 0.05s
✓ an event with no link shows no join button                                    0.06s
✓ a malformed meeting link does not crash or render a button                    0.06s
✓ a javascript url is never rendered as a join button                           0.05s
✓ a student does not see the names of their classmates                          0.06s

 PASS  Tests\Feature\CourseEnrolmentTest
✓ a student cannot leave a course themselves                                    0.05s
✓ the owning lecturer can remove a student                                      0.04s
✓ a lecturer cannot remove a student from another lecturers course              0.05s
✓ a student cannot remove a classmate                                           0.04s
✓ removing somebody who is not enrolled is a 404                                0.04s

 PASS  Tests\Feature\CourseMaterialUploadTest
✓ a lecturer can upload a normal document                                       0.06s
✓ a php file is refused                                                         0.04s
✓ a php file renamed to look like a pdf is still refused                        0.05s
✓ an html file is refused                                                       0.04s
✓ a javascript url cannot be saved as an external material                      0.04s
✓ an ordinary external link is accepted                                         0.04s
✓ a lecturer cannot upload to another lecturers course                          0.04s

 PASS  Tests\Feature\ExampleTest
✓ the application returns a successful response                                 0.04s

 PASS  Tests\Feature\ProfileTest
✓ profile page is displayed                                                     0.06s
✓ profile information can be updated                                            0.04s
✓ email verification status is unchanged when the email address is unchanged    0.04s
✓ user can delete their account                                                 0.03s
✓ correct password must be provided to delete account                           0.03s
```

```
 PASS  Tests\Feature\ProfileTest
✓ profile page is displayed                                                     0.06s
✓ profile information can be updated                                            0.04s
✓ email verification status is unchanged when the email address is unchanged    0.04s
✓ user can delete their account                                                 0.03s
✓ correct password must be provided to delete account                           0.03s

 PASS  Tests\Feature\SubjectExpertBadgeTest
✓ passing every quiz in the subject awards the badge                            0.06s
✓ attempting without passing does not award it                                  0.04s
✓ a subject with no quizzes awards nothing                                      0.03s
✓ resubmitting does not award a second copy                                     0.05s
✓ a quiz added afterwards does not revoke the badge                             0.05s
✓ the badge is scoped to its own subject                                        0.06s

 PASS  Tests\Feature\WebServiceTest
✓ every response carries status timestamp and the request id                    0.05s
✓ a request without the mandatory ifa fields is refused                         0.03s
✓ an internal service refuses a caller with no api key                          0.03s
✓ an internal service refuses a wrong api key                                   0.03s
✓ the credential service is public and needs no key                             0.43s
✓ detail flag one does not disclose the holder                                  0.45s
✓ detail flag two returns the holder                                            0.54s
✓ an unknown credential reports not found rather than erroring                  0.03s
✓ a tampered credential is reported over the service                            0.45s
✓ the quiz service returns the best attempt                                     0.06s
✓ the analytics service returns cohort figures only                             0.05s
✓ the notification service writes to the inbox                                  0.03s
✓ the notification service refuses a type outside the allow list                0.03s
✓ module 1 consumes module 2s course info service                               0.08s
✓ module 5 consumes module 1s credential service                                0.42s
✓ module 3 consumes module 4s quiz result service                               0.05s
✓ a client returns null rather than throwing when the service is unreachable    1.07s

Tests:    86 passed (200 assertions)
Duration: 13.37s
```

Figure 8.1: Terminal Output of php artisan test Showing 86 Passing Tests

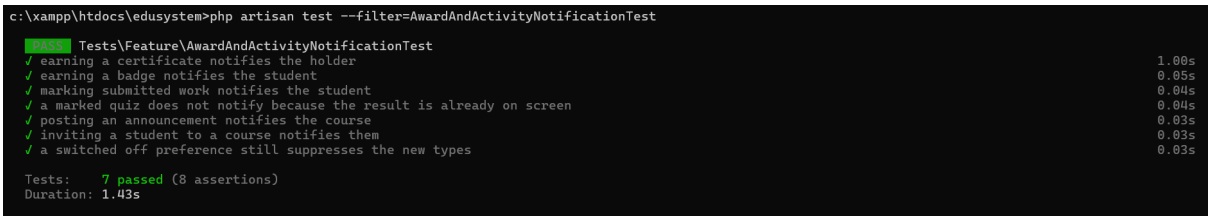



Figure 8.2: Terminal Output of own modules php artisan test

## Appendix B: GitHub Repository URL

Team Repository: https://github.com/NickFoo0924/edusystem

Branch: master